

Посібник з навчання

Демо FMI-скіда — зберіть, запустіть, навчіть і отримайте готову наглядову мережу

Повний покроковий маршрут для копіювання навчальної половини конвеєра: встановіть інструментарій, зберіть платформу, запустіть сценарії скіда, навчіть чотири діагностичні голови й отримайте готову до розгортання наглядову мережу, яку споживає Посібник з розгортання.

Документ	Посібник з навчання моделі
Продукт	Jneopallium Industrial Loop Guardian (демо FMI-скіда)
Модель	industrial-loop-guardian 1.0.0
Тренер	scripts/demo-industrial-fmi/ train_loop_guardian_model.py
Вихід	Готова до розгортання конфігурація JNeopallium
Автор	Дмитро Раковський — Харків, Україна
Дата	23 червня 2026
Ліцензія	BSD 3-Clause

Зміст

1. Чого ви досягнете
2. Де закінчується навчання і починається розгортання
3. Передумови та системні вимоги
4. Крок 1 — Отримати код і зібрати
5. Крок 2 — Запустити сценарії FMI-скіда
6. Крок 3 — Зрозуміти згенеровані результати
7. Крок 4 — Навчити модель Loop Guardian
8. Крок 5 — Швидка перевірка моделі на Python
9. Крок 6 — Згенерувати звіти про навчання та запуски
10. Крок 7 — Згенеровані артефакти: готова до розгортання мережа
11. Крок 8 — Перевірити запуск за контрольними критеріями
12. Усунення несправностей
13. Додаток — шпаргалка навчання

1 Чого ви досягнете

До кінця цього посібника ви матимете: зібрану платформу Jneopallium; детерміновані запуски FMI-скіда за дев'ятьма сценаріями; свіжонавчену модель обслуговування й енергії **Industrial Loop Guardian**; і — ключовий результат — **повну, готову до розгортання мережу JNeopallium** (п'ять файлів рівнів, справжні класи часу виконання, вбудовані навчені голови, фіксований запобіжник і готовий промисловий контекст), яку напряду завантажує супровідний **Посібник з розгортання**.

Стеля безпеки для всього конвеєра

Усе, що робить навчена модель, є наглядовим і ADVISORY. Керування PLC/PID/SIS лишається детермінованим і авторитетним; тренер це кодує — рівень відповіді ADVISORY, жорсткі запобіжники — фіксована конфігурація, а записи виконавчих механізмів OPC UA лишаються окремими.

2 Де закінчується навчання і починається розгортання

Фаза	Що відбувається	Документ
Навчання (цей посібник)	Зборка, запуск сценаріїв, навчання, видача мережі	Посібник з навчання
Розгортання	Завантаження виданої мережі, налаштування джерел подій, запуск worker	Посібник з розгортання

Передача — це єдина тека згенерованих артефактів (Крок 7). Навчання її **створює**; розгортання її **споживає**. Тренер записує готову для JNeopallium конфігурацію рівнів і нейронів *і* файл промислового контексту, тож ті самі файли, що фіксують навчання, є файлами, з яких завантажується worker.

3 Передумови та системні вимоги

Інструментарій

Інструмент	Версія	Навіщо
Java JDK	17 або новіша (LTS)	Зборка й запуск worker / контролера
Apache Maven	3.9 або новіша	Багатомодульна збірка master + worker
Python	3.10+ (тестовано 3.13)	Запускає тренер, раннер демо й генератор звітів
Git	будь-яка свіжа	Клонування репозиторію
Docker + Mosquitto (опційно)	будь-яка свіжа	Лише для повного шляху OPC UA + MQTT

Типовий раннер «однієї команди» дає детерміновані докази **без** Docker, брокера чи сервера OPC UA. Повний протокольний шлях (шлюз + IndustrialFmiDemoMain) додатково потребує Maven, Mosquitto/Docker, залежностей Python-шлюзу та зібраного FMU.

Перевірте інструментарій

```
java -version      # очікуємо 17 або новіше
mvn -version       # очікуємо 3.9 або новіше
python --version   # очікуємо 3.10 або новіше
```

4 Крок 1 — Отримати код і зібрати

```
git clone https://github.com/rakovpublic/jneopallium.git
cd jneopallium
mvn clean install
```

Збірка встановлює `com.rakovpublic.jneopallium:worker:1.0`. Worker уже містить справжні класи промислових нейронів, процесорів і сигналів, на які посилається навчена мережа (`com.rakovpublic.jneopallium.worker.net.neuron.impl.industrial.*` та інші), тож для навчання не потрібні додаткові написані вручну класи.

5 Крок 2 — Запустити сценарії FMI-скіда

Детермінований раннер просуває змодельований скід і записує докази для кожного сценарію. Запустіть усі дев'ять або по одному:

Linux / macOS

```
scripts/demo-industrial-fmi/run_demo.sh all
scripts/demo-industrial-fmi/run_demo.sh pump-wear
scripts/demo-industrial-fmi/run_demo.sh high-temperature-interlock
```

Windows (PowerShell)

```
powershell -ExecutionPolicy Bypass -File scripts/demo-industrial-fmi/run_demo.ps1 all
```

Дев'ять сценаріїв: `normal`, `load-disturbance`, `oscillation`, `pump-wear`, `temperature-sensor-drift`, `mqtt-outage`, `opcua-outage`, `high-temperature-interlock` та `operator-override`. Кожен вправляє іншу частину наглядової та безпекової поведінки.

6 Крок 3 — Зрозуміти згенеровані результати

Кожен запуск записує теку для сценарію під `target/`:

```
target/jneopallium-industrial-fmi/<scenario>/
manifest.json          process_trace.csv          controller_results.jsonl
advisory_findings.jsonl model_advisory_findings.jsonl
heuristic_advisory_findings.jsonl
opcua_audit.jsonl      mqtt_audit.jsonl          alarms.jsonl
```

Файл	Що містить
metrics.json	KPI сценарію: IAE, перерегулювання, час усталення, енергія, затримка інтерлока, доступність
comparison.json	Порівняння базової лінії та Jneopallium
model_advisory_findings.jsonl	Рекомендації від навченої моделі
advisory_findings.jsonl	Рекомендації з нейроном-власником, економічним базисом, межею безпеки, межею
opcua_audit.jsonl / mqtt_audit.jsonl	Кожна подія команди/телеметрії, для межі протоколів

7 Крок 4 — Навчити модель Loop Guardian

Тренер не потребує сторонніх пакетів Python. Навчіть і експортуйте модель обслуговування й енергії командою еталона промислового масштабу:

```
python scripts/demo-industrial-fmi/train_loop_guardian_model.py \
--reference-multiplier 1000 \
--target-corpus-bytes 100gb \
--max-corpus-bytes 100gb
```

Він записує повний набір артефактів у `worker/src/main/resources/model/industrial-loop-guardian/` (див. Крок 7). Тренер припасовує чотири діагностичні голови над 39 інженерними ознаками, оцінює їх на розбитті без витоку й друкує зведення з точністю/повнотою/F1 для кожного висновку.

Що означають прапорці

--reference-multiplier детерміновано розширює вбудований корпус скіда. --target-corpus-bytes фіксує логічну ціль промислового масштабу (100 ГіБ), досягнуту реплікацією. --max-corpus-bytes — жорсткий запобіжник, що переривається, замість роздувати репозиторій — припасована вибірка лишається кілька десятків МіБ, тож запуск завершується на робочій станції.

Підсистема моніторингу стану машини

Поряд із чотирма головами обслуговування й енергії оновлена модель містить мультимодальну підсистему **здоров'я машини** (акустика + вібрація), реалізовану як рівень часу виконання Java. Її екстрактори ознак, нормалізація робочого режиму, оцінка базової лінії / зсуву домену, гіпотези несправностей і запобіжник рекомендацій призначені для навчання й валідації на публічних наборах моніторингу стану:

Набір даних	Роль
MIMII	Звук промислових машин у нормальному й несправному станах
DCASE machine-condition	Бенчмарк виявлення акустичних аномалій

MIMII DUE / DG	Варіанти зсуву домену / узагальнення домену
Paderborn Bearing Data Center	Розмічені вібраційні дані несправностей підшипників
Симулятор насоса FMI	Вбудовані детерміновані сигнали скіда для відтворюваних доказів
Авторизована телеметрія майданчика	Ваші власні машини після перегляду обробки даних

Підсистема **лише для читання / тінь / рекомендації** за побудовою: вона видає `MachineHealthAdvisorySignal` (бал здоров'я, ймовірності несправностей, невідома аномалія, зсув домену, невпевненість) і ніколи не записує команду виконавчого механізму. Виходи зсуву домену й невпевненості дають їй казати «я не впевнений» на незнайомій машині, а не перебільшувати — саме цього вимагає реальна валідація моніторингу стану перед будь-якою промисловою опорою.

8 Крок 5 — Швидка перевірка моделі на Python

Для швидкої перевірки логіки моделі без збірки запустіть модульні тести Python:

```
python -m unittest discover -s scripts/demo-industrial-fmi/tests
```

Вони проганяють тренер і модель `loop-guardian` наскрізно без Maven, брокера чи зібраного FMU — ідеально для швидкого зворотного зв'язку під час ітерацій над ознаками чи корпусом.

9 Крок 6 — Згенерувати звіти про навчання та запуски

Після навчання й відтворення згенеруйте зведені звіти:

```
python scripts/demo-industrial-fmi/generate_run_reports.py
```

Це збирає метрики кожного сценарію та докази навченої моделі у відтворювані звіти, які можна додати до пілотної оцінки.

10 Крок 7 — Згенеровані артефакти: готова до розгортання мережа

Це передача конвеєра. Запуск навчання записує теку конфігурації у стилі JNeopallium, готову до розгортання як є, — власне мережу рівнів і нейронів, з якої завантажується worker, плюс готовий промисловий контекст:

Артефакт	Що це
<code>trained-industrial-loop-guardian-model.json</code>	Компактна навчена модель: ознаки, чотири голови, ваги, метрики
<code>model-descriptor.json</code>	Опис усієї мережі: 5 рівнів, 17 нейронів, мапа частот, класи часу виконання
<code>layer-0.json</code>	Межа входу заводу + наглядного контексту
<code>layer-1-fast-telemetry.json</code>	Валідація швидкої телеметрії і стан контуру — 7 нейронів

layer-2-maintenance-energy.json	Чотири навчені діагностичні голови висновків
layer-3-advisory-planning.json	Економічне планування рекомендацій + фіксований запобіжник — 4 нейрони
result-layer.json	Вивід промислових рекомендацій JSONL
trained-model-update.json	Останній знімок навчання: статус, контрольна сума, ваги, зведення
quantitative-summary.json	Масштаб, метрики та найвагоміші ваги ознак на голову
source-mapping.json	На які типізовані сигнали відображається кожне джерело даних
production-context.json	Готовий контекст рекомендаційного worker для розгортання

Мережа, яку будує тренер

model-descriptor.json фіксує п'ятирівневу мережу з **17 справжніх нейронів**, з **156 навчуваними вагами** та **4 зсувами**, де кожен нейрон посилається на конкретний клас із промислового пакета worker:

Рівень	Розмір	Роль
0 — Вхід	0	Межа заводу / OPC UA / MQTT / FMI-відтворення / Kafka
1 — Швидка телеметрія	7	Валідація, інтерлоки, перевизначення, стан контуру
2 — Діагностичні голови	4	Знос насоса, коливання/заклинювання, дрейф датчика, енергія
3 — Планування рекомендацій	4	Планування обслуговування, обмежений трим, економіка, запобіжник
4 — Результат	2	Рекомендації з обслуговування та енергії як JSONL

Кожна діагностична голова вбудовує свої навчені ваги, дозволений їй фільтр ознак (голова коливань, наприклад, обмежена 17 із 39 ознак) та свою логічну роль і власне міркування. Опис також містить signalFrequencyMap, що дає кожному сигналу його каданс — швидкі теги щоцикл, деградація/енергія/установка кожні 10, вікна обслуговування кожні 60 — багаторасовий дизайн, втілений конкретно.

Чому це важливо

Оскільки навчання видає справжню, придатну до перевірки, готову до розгортання конфігурацію, між «тим, що ми навчили» і «тим, що ми запускаємо», немає кроку перекладу. Посібник з розгортання просто спрямовує worker на цю теку й завантажує production-context.json.

11 Крок 8 — Перевірити запуск за контрольними критеріями

1. Усі очікувані родини джерел присутні в `source-mapping.json` (FMI-відтворення, OPC UA, MQTT, тіньовий Kafka, CMMS, лічильник енергії).
2. F1 кожного висновку на рівні або близько 1.0, а рівень хибних спрацювань енергетичної голови в межах бюджету (еталонний запуск: $\text{macro-F1} \approx 0.9995$, macro рівень хибних ≈ 0.0005).
3. П'ять файлів рівнів, `trained-model-update.json` і `production-context.json` записано, і вони посилаються на очікувані класи часу виконання.
4. Кожен сценарій дав детерміновані траси й порівняння з базовою лінією.
5. Рекомендаційний JSON містить нейрон-власник, код рекомендації, економічний базис, результат межі безпеки та межі PLC/PID/SIS проти Jneopallium.

Перевірка відтворюваності

Еталонний запуск детермінований. Повторний запуск тієї самої команди має дати ту саму контрольну суму (`trained-model-update.json`) і ті самі метрики на голову. Розбіжність — сигнал, що змінилися вхідні дані чи інструментарій.

12 Усунення несправностей

Симптом	Імовірна причина й виправлення
<code>java -version</code> показує <code>< 17</code>	Встановіть JDK 17+ і вкажіть <code>JAVA_HOME</code>
Тренер не знаходить сценарії	Запускайте з кореня репозиторію; сценарії в <code>scripts/demo-industrial-fmi/config/scenarios /</code>
«estimated canonical corpus exceeds --max-corpus-bytes»	Знизьте <code>--reference-multiplier/--target-corpus-bytes</code> або свідомо підніміть запобіжник
Помилки FMU / шлюзу	Типовий раннер їх не потребує; лише повний шлях потребує FMU + брокер
Файли рівнів не записано	Переконайтеся, що <code>--output-dir</code> доступний для запису; тренер пише туди всі артефакти
Модульні тести падають після зміни	Зміна ознаки чи корпусу змінила поведінку; перегляньте diff та очікувані метрики

13 Додаток — шпаргалка навчання

Збірка

```
git clone https://github.com/rakovpublic/jneopallium.git && cd jneopallium && mvn clean install
```

Запуск сценаріїв

```
scripts/demo-industrial-fmi/run_demo.sh all # Linux/macOS
```



```
powershell -File scripts/demo-industrial-fmi/run_demo.ps1 all # Windows
```

Навчання, перевірка, звіти

```
python scripts/demo-industrial-fmi/train_loop_guardian_model.py \  
  --reference-multiplier 1000 --target-corpus-bytes 100gb --max-corpus-bytes 100gb  
  
python -m unittest discover -s scripts/demo-industrial-fmi/tests  
python scripts/demo-industrial-fmi/generate_run_reports.py
```

Jneopallium Industrial Loop Guardian · Посібник з навчання. Тренер видає готову до розгортання мережу JNeopallium плюс промисловий контекст; як її запустити — у Посібнику з розгортання. Наглядовий над PLC/PID/SIS. Режим безпеки: ADVISORY. Ліцензія: BSD 3-Clause.